Codex Coding Prompts

Prompts designed specifically for this coding app so it inspects the repo, edits the right files, and verifies changes with a higher first-attempt success rate.

How To Get Better Results In This App

- Name the exact behavior you want changed, not just the general area.
- Give expected behavior and constraints like backwards compatibility or style matching.
- Tell Codex whether you want a fix, feature, refactor, review, or explanation.
- Mention which tests or commands matter if you already know them.
- Ask for verification, not just code edits.

Best results usually come from prompts that tell Codex to inspect first, keep changes scoped, follow repo conventions, and verify after editing.

Copy-And-Use Prompt Library

Best Default Codex Prompt

Use this when you want Codex to inspect the repo, implement the change, and verify it with minimal back-and-forth.

```
Work in this codebase and implement [feature / fix / refactor].
Start by inspecting the relevant files and understanding the current behavior before changing code.
Then make the smallest correct change that solves the problem.
After editing, run the most relevant checks or tests you can.
In your final response, give me:
- what changed
- any important tradeoffs or assumptions
- how you verified it
```

Bug Fix Prompt

Best for reproducible bugs, regressions, and broken flows.

```
Fix this bug in the current repo.
Problem: [describe bug].
Expected behavior: [what should happen].
If the root cause is not obvious, inspect the code path first and infer the most likely cause from the implementation.
Then patch it directly, keep the fix scoped, and add or update tests if appropriate.
Finish by telling me the root cause, the fix, and the verification steps you ran.
```

Feature Implementation Prompt

For adding a new endpoint, component, workflow, CLI flag, or behavior.

```
Implement this feature in the existing codebase: [feature].
Requirements:
- preserve existing patterns and architecture
- avoid unrelated refactors
- update tests, types, and docs only where needed
- keep the UX / API intuitive and backwards compatible unless stated otherwise
Please inspect the existing implementation style first, then make the change end to end.
```

Code Review Prompt

For review-style feedback instead of editing.

```
Review the current changes like a senior engineer.
Focus on bugs, regressions, edge cases, risky assumptions, and missing tests.
Prioritize findings over summary.
For each finding, include the file path, the problem, why it matters, and the likely impact.
If no serious issues are found, say that explicitly and mention any remaining test gaps or risks.
```

Refactor Prompt

For cleanup work where behavior should stay the same.

```
Refactor the relevant code for [goal: readability / maintainability / duplication reduction / structure].
Do not change external behavior unless absolutely necessary.
First identify the current pattern and constraints in the codebase.
Then make the refactor incrementally, keep it easy to review, and run the most relevant verification afterward.
Call out any risky areas where a larger follow-up might be needed.
```

Add Tests Prompt

For strengthening confidence around existing behavior or a recent bug fix.

```
Add or improve tests for [module / behavior / bug].
Inspect the current test style first and match it.
Cover the most important happy path, edge case, and regression scenario without adding unnecessary noise.
If the code is hard to test, explain the smallest production change needed to make testing practical, then implement it if it is low risk.
```

Debugging Prompt

For errors that are unclear or need investigation before a fix.

```
Investigate this issue in the repo: [issue].
Do not guess immediately.
Trace the likely code path, inspect the relevant files, and explain the most probable root cause based on the implementation.
If confidence is high, fix it and verify the result.
If confidence is low, narrow it down as far as possible and tell me the next highest-value check.
```

Performance Prompt

For latency, slow queries, rendering cost, build slowness, or hot-path cleanup.

```
Improve the performance of [feature / path / query / component].
First identify the likely bottleneck from the code and existing architecture.
Then implement the highest-leverage improvement that keeps complexity reasonable.
Avoid premature micro-optimizations.
In the final answer, summarize the bottleneck, the optimization, and any tradeoffs or measurement limits.
```

Frontend Prompt

For UI changes where you want Codex to preserve the current design system instead of inventing a new one.

```
Implement this frontend change in the existing app: [change].
Inspect the current UI patterns, component conventions, styling approach, and state management first.
Match the established design language unless I explicitly ask for a redesign.
Make the UI responsive and production-ready, and verify the changed flow if a local route or page is obvious.
```

Backend / API Prompt

For routes, services, validation, auth, jobs, or data-layer work.

```
Implement this backend change: [change].
Please inspect the relevant routes, services, models, validators, and tests first.
Preserve existing error handling and naming conventions.
Include any necessary schema, validation, or test updates, but avoid broad cleanup outside the task.
Verify with the most relevant targeted checks.
```

Repo Onboarding Prompt

For understanding an unfamiliar codebase quickly.

```
Orient me to this repository.
I want to understand:
- the app structure
- the main entry points
- the key data flow / request flow
- where I would change [specific area]
Base your explanation on the actual files in the repo, and cite the most relevant paths as you walk through it.
```

PR / Change Summary Prompt

For generating a clean summary after code has already changed.

```
Summarize the current code changes in a way I can use for a PR description or update.
Please include:
- what changed
- why it changed
- any user-visible impact
- how it was verified
Keep it concise, concrete, and grounded in the actual diff.
```

One-Line Booster

Append this to almost any coding request in Codex: "Inspect the relevant code first, make the smallest correct change, and run the most relevant verification before finalizing."